

/proc is your friend

Live Forensics - Investigating a Compromised Linux System

Author: Adli Wahid adli@apnic.net

Credits to Craig Rowland (Sandfly Security)

Basic Linux Malware Forensics

We'll perform a step-by-step analysis of a *bind shell backdoor* waiting for a connection on an Ubuntu Server. In this section, you'll understand the significance of running processes for investigation. These information are volatile as well, such that they will be gone if the computer is turned off. In other words, you will not get some of these information from the physical disk imaging.

Login to the Lubuntu desktop and use the terminal for the following exercise.

1. Simulating the attack. In this case we will use Netcat (nc) to open a TCP shell on port 31337. We'll make it send data to /dev/null instead of giving shell.

```
cd /tmp
cp /bin/nc /tmp/x7
./x7 -vv -k -w 1 -l 31337 > /dev/null &
rm x7
```

2. In the above we made a copy of the netcat binary, renamed it to x7 and made it listen on port 31337. Then we delete x7!

3. Verify that x7 is running. Please take note of the process id (number)

- ps aux | grep x7
- Note: The process id will be unique to your vm/server.
- Sample output:

```
ps aux | grep x7
```

Example Output:

```
adli      2247  0.0  0.0   9180   952 pts/0    S      10:38   0:00 ./x7 -vv -k -w 1 -l 31337
```

4. Check for the network activities

- sudo netstat -punat
- Note: netstat shows a process named 'x7' PID with a listening port that we don't recognize.
- Sample output:

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN	2085/sshd
tcp	0	0	0.0.0.0:31337	0.0.0.0:*	LISTEN	2247/x7
tcp	0	296	192.168.56.109:22	192.168.56.1:51034	ESTABLISHED	2196/sshd: adli [pr
udp	0	0	0.0.0.0:68	0.0.0.0:*		894/dhclient]

5. When doing live forensics on Linux, /proc is your best friend! A lot of information related to running processes are kept in /proc/\$PID

- take note of the PID (process id) of x7 on your system and run:
- `ls -al /proc/$PID` ← replace PID with your own PID!
- Note: We'll see a few interesting things: (1) The current working directory (cwd) is /tmp (2) the binary was in /tmp but was deleted

Sample output:

```
adli@ubuntubox-1604:/tmp$ ls -al /proc/2247
total 0
dr-xr-xr-x  9 adli adli 0 May 19 10:39 .
dr-xr-xr-x 129 root root 0 May 19 10:34 ..
dr-xr-xr-x  2 adli adli 0 May 19 10:55 attr
-rw-r--r--  1 adli adli 0 May 19 10:55 autogroup
-r-----  1 adli adli 0 May 19 10:55 auxv
-r--r--r--  1 adli adli 0 May 19 10:55 cgroup
--w-----  1 adli adli 0 May 19 10:55 clear_refs
-r--r--r--  1 adli adli 0 May 19 10:39 cmdline
-rw-r--r--  1 adli adli 0 May 19 10:55 comm
-rw-r--r--  1 adli adli 0 May 19 10:55 coredump_filter
-r--r--r--  1 adli adli 0 May 19 10:55 cpuset
lrwxrwxrwx  1 adli adli 0 May 19 10:55 cwd -> /tmp
-r-----  1 adli adli 0 May 19 10:55 environ
lrwxrwxrwx  1 adli adli 0 May 19 10:55 exe -> /tmp/x7 (deleted)
dr-x-----  2 adli adli 0 May 19 10:39 fd
dr-x-----  2 adli adli 0 May 19 10:55 fdinfo
-rw-r--r--  1 adli adli 0 May 19 10:55 gid_map
-r-----  1 adli adli 0 May 19 10:55 io
-r--r--r--  1 adli adli 0 May 19 10:55 limits
-rw-r--r--  1 adli adli 0 May 19 10:55 loginuid
dr-x-----  2 adli adli 0 May 19 10:55 map_files
-r--r--r--  1 adli adli 0 May 19 10:55 maps
-rw-----  1 adli adli 0 May 19 10:55 mem
-r--r--r--  1 adli adli 0 May 19 10:55 mountinfo
-r--r--r--  1 adli adli 0 May 19 10:55 mounts
-r-----  1 adli adli 0 May 19 10:55 mountstats
dr-xr-xr-x  5 adli adli 0 May 19 10:55 net
dr-x--x--x  2 adli adli 0 May 19 10:55 ns
-r--r--r--  1 adli adli 0 May 19 10:55 numa_maps
-rw-r--r--  1 adli adli 0 May 19 10:55 oom_adj
-r--r--r--  1 adli adli 0 May 19 10:55 oom_score
-rw-r--r--  1 adli adli 0 May 19 10:55 oom_score_adj
-r-----  1 adli adli 0 May 19 10:55 pagemap
-r-----  1 adli adli 0 May 19 10:55 personality
-rw-r--r--  1 adli adli 0 May 19 10:55 projid_map
lrwxrwxrwx  1 adli adli 0 May 19 10:55 root -> /
-rw-r--r--  1 adli adli 0 May 19 10:55 sched
-r--r--r--  1 adli adli 0 May 19 10:55 schedstat
-r--r--r--  1 adli adli 0 May 19 10:55 sessionid
-rw-r--r--  1 adli adli 0 May 19 10:55 setgroups
-r--r--r--  1 adli adli 0 May 19 10:55 smaps
-r-----  1 adli adli 0 May 19 10:55 stack
-r--r--r--  1 adli adli 0 May 19 10:39 stat
-r--r--r--  1 adli adli 0 May 19 10:55 statm
-r--r--r--  1 adli adli 0 May 19 10:39 status
```

```
-r----- 1 adli adli 0 May 19 10:55 syscall
dr-xr-xr-x 3 adli adli 0 May 19 10:55 task
-r--r--r-- 1 adli adli 0 May 19 10:55 timers
-rw-r--r-- 1 adli adli 0 May 19 10:55 uid_map
-r--r--r-- 1 adli adli 0 May 19 10:55 wchan
```

- Note: A lot of exploits work out of /tmp and /dev/shm on Linux. These are both world writable directories on almost all Linux systems and many malware and exploits will drop their payloads there to run. A process that is making its home in /tmp or /dev/shm is suspicious.

6. Now we will try to recover the deleted binary. As long process is still running, it is very easy to recover a deleted process binary on Linux:

- `cp /proc/$PID/exe /tmp/recovered_bin <- REPLACE PID with your own`
- Note: basically /proc/\$PID/exe is a copy of the running executable. This makes it worthwhile to recover the binary and save it somewhere before you kill the process or shutdown the machine. Of course, we need to be careful

7. Now we will validate the copied file with the original binary (Netcat)

- `md5sum recovered_bin /bin/nc`
- Note: compare the hash. In addition to md5sum, you can also use sha256sum. If both hashes are the same then we are looking at the same binaries/files

```
adli@ubuntubox-1604:/tmp$ sha256sum recovered_bin /bin/nc
92a61a6f9f689751d0de8099e73cd23d8c28c25331ae9d5c21dcd325970390cb recovered_bin
92a61a6f9f689751d0de8099e73cd23d8c28c25331ae9d5c21dcd325970390cb /bin/nc
```

8. We will now explore the commands (name & full parameters) of the process. Some malware will give itself a vague or closely to legit process names (i.e. apache or sshd), so we can check both /proc/PID/command/proc/PID/cmdline

- `cat /proc/$PID/comm`
- `cat /proc/$PID/cmdline`
- Note: in our case both results are x7 but we can see the parameters that was used when issuing the command

```
$cat /proc/$PID/comm
x7
```

```
adli@ubuntubox-1604:/tmp$ cat /proc/$PID/cmdline && echo "\n"
./x7-vv-k-w1-l31337\n
```

9. Malware process environment. Now let's take a look at the environment our malware inherited when it started. This can often reveal information about who or what started the process. Here we see the process was started with sudo by another user:

- `strings /proc/$PID/envron`
- Note: have a look at the output as it provides information on users, location, directory path, etc
- Sample output:

```
adli@ubuntubox-1604:/tmp$ strings /proc/$PID/envron
XDG_SESSION_ID=3
TERM=screen
SHELL=/bin/bash
SSH_CLIENT=192.168.56.1 51034 22
SSH_TTY=/dev/pts/0
USER=adli
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01
MAIL=/var/mail/adli
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/b
PWD=/tmp
```

```

LANG=en_US.UTF-8
SHLVL=1
HOME=/home/adli
LOGNAME=adli
SSH_CONNECTION=192.168.56.1 51034 192.168.56.109 22
LESSOPEN=| /usr/bin/lesspipe %s
XDG_RUNTIME_DIR=/run/user/1000
LESSCLOSE=/usr/bin/lesspipe %s %s
_=./x7
OLDPWD=/home/adli

```

10. We can also see the file descriptors that the malware has open. This can often show you hidden files and directories that the malware is using to stash things along with open sockets:

- `ls -al /proc/$PID/fd`
- Note: This shows that the open file descriptors that process has
- Note: You can see that 0,1,2 stdin/stdout/stderr. There is also an open socket

```

adli@ubuntubox-1604:/tmp$ ls -lah /proc/$PID/fd
total 0
dr-x----- 2 adli adli  0 May 19 10:39 .
dr-xr-xr-x 9 adli adli  0 May 19 10:39 ..
lrwx----- 1 adli adli 64 May 19 10:46 0 -> /dev/pts/0
l-wx----- 1 adli adli 64 May 19 10:46 1 -> /dev/null
lrwx----- 1 adli adli 64 May 19 10:39 2 -> /dev/pts/0
lrwx----- 1 adli adli 64 May 19 10:46 3 -> socket:[18469]

```

11. Another area to look into is the Linux process maps. This shows libraries the malware is using and again can show links to malicious files it is using as well.

- `cat /proc/$PID/maps`
- Note: The `/proc/$PIDmaps` file will show libraries and other data the binary has open. It can help reveal hidden areas where the process is stashing data or libraries that are malicious.
- Sample output:

```

adli@ubuntubox-1604:/tmp$ cat /proc/$PID/maps
00400000-00407000 r-xp 00000000 fc:00 1333 /tmp/x7 (deleted)
00606000-00607000 r--p 00006000 fc:00 1333 /tmp/x7 (deleted)
00607000-00608000 rw-p 00007000 fc:00 1333 /tmp/x7 (deleted)
00608000-00688000 rw-p 00000000 00:00 0
01dfa000-01e1b000 rw-p 00000000 00:00 0 [heap]
7fb8305d2000-7fb830792000 r-xp 00000000 fc:00 35835 /lib/x86_64-linux-gnu/libc-2
7fb830792000-7fb830992000 ---p 001c0000 fc:00 35835 /lib/x86_64-linux-gnu/libc-2
7fb830992000-7fb830996000 r--p 001c0000 fc:00 35835 /lib/x86_64-linux-gnu/libc-2
7fb830996000-7fb830998000 rw-p 001c4000 fc:00 35835 /lib/x86_64-linux-gnu/libc-2
7fb830998000-7fb83099c000 rw-p 00000000 00:00 0
7fb83099c000-7fb8309b3000 r-xp 00000000 fc:00 35831 /lib/x86_64-linux-gnu/libres
7fb8309b3000-7fb830bb3000 ---p 00017000 fc:00 35831 /lib/x86_64-linux-gnu/libres
7fb830bb3000-7fb830bb4000 r--p 00017000 fc:00 35831 /lib/x86_64-linux-gnu/libres
7fb830bb4000-7fb830bb5000 rw-p 00018000 fc:00 35831 /lib/x86_64-linux-gnu/libres
7fb830bb5000-7fb830bb7000 rw-p 00000000 00:00 0
7fb830bb7000-7fb830bca000 r-xp 00000000 fc:00 1288 /lib/x86_64-linux-gnu/libbsd
7fb830bca000-7fb830dc9000 ---p 00013000 fc:00 1288 /lib/x86_64-linux-gnu/libbsd
7fb830dc9000-7fb830dca000 r--p 00012000 fc:00 1288 /lib/x86_64-linux-gnu/libbsd
7fb830dca000-7fb830dcb000 rw-p 00013000 fc:00 1288 /lib/x86_64-linux-gnu/libbsd
7fb830dcb000-7fb830dcc000 rw-p 00000000 00:00 0
7fb830dcc000-7fb830df2000 r-xp 00000000 fc:00 35827 /lib/x86_64-linux-gnu/ld-2.2
7fb830fe7000-7fb830feb000 rw-p 00000000 00:00 0
7fb830ff1000-7fb830ff2000 r--p 00025000 fc:00 35827 /lib/x86_64-linux-gnu/ld-2.2

```

```

7fb830ff2000-7fb830ff3000 rw-p 00026000 fc:00 35827 /lib/x86_64-linux-gnu/ld-2.23
7fb830ff3000-7fb830ff4000 rw-p 00000000 00:00 0
7ffe41846000-7ffe41867000 rw-p 00000000 00:00 0 [stack]
7ffe41880000-7ffe41882000 r--p 00000000 00:00 0 [vvar]
7ffe41882000-7ffe41884000 r-xp 00000000 00:00 0 [vdso]
ffffffff600000-ffffffff601000 r-xp 00000000 00:00 0 [vsyscall]

```

12. The `/proc/$PID/stack` area can sometimes reveal more details. We'll look at that like this:

- `sudo cat /proc/$PID/stack`
- Note: Note that this file provides additional insights on what the malware/process might be doing. These `accept()` calls indicate that it is waiting for an inbound network connection. If a process is hiding the network port from `netstat`, you can still see what it is doing here.
- Sample output:

```

[<ffffffff817650c1>] inet_csk_accept+0x271/0x350
[<ffffffff817938cc>] inet_accept+0x3c/0x130
[<ffffffff816fd563>] SYSC_accept4+0x103/0x210
[<ffffffff816ff150>] SyS_accept+0x10/0x20
[<ffffffff818244f2>] entry_SYSCALL_64_fastpath+0x16/0x71
[<ffffffffffffffff>] 0xffffffffffffffff

```

13. Finally, let's look at `/proc/$PID/status` for overall process details. This can reveal parent PIDs and so forth.

- `cat /proc/$PID/status`
- Note: This file gives information on parent PIDs (if any), memory usage, etc
- Sample output:

```
adli@ubuntu1604:/tmp$ cat /proc/$PID/status
```

```

Name:   x7
State:  S (sleeping)
Tgid:   2247
Ngid:   0
Pid:    2247
PPid:   2230
TracerPid: 0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 110 115 116 1000
NSStgid: 2247
NSpid:  2247
NSpgid: 2247
NSsid:  2230
VmPeak: 9180 kB
VmSize: 9180 kB
VmLck:  0 kB
VmPin:  0 kB
VmHWM:  952 kB
VmRSS:  952 kB
VmData: 708 kB
VmStk:  136 kB
VmExe:   28 kB
VmLib:  2112 kB
VmPTE:   40 kB
VmPMD:   12 kB
VmSwap:  0 kB
HugetlbPages: 0 kB
Threads: 1

```

```
SigQ: 0/3819
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: 0000000000000000
SigCgt: 0000000000000000
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 0000003fffffffff
CapAmb: 0000000000000000
Seccomp: 0
Cpus_allowed: 1
Cpus_allowed_list: 0
Mems_allowed: 00000000,00000001
Mems_allowed_list: 0
voluntary_ctxt_switches: 3
nonvoluntary_ctxt_switches: 2
```

14. Discussion / Let's Recap

- What is the value of Live Forensics
- What have you learned?
- Don't kill a suspicious process until you have investigated what it is doing
- Have this cheatsheet with you : <https://www.sandflysecurity.com/blog/compromised-linux-cheat-sheet/>
- Questions?

eof.